

Assignment 3: LibraFlow — Digital Library System Pending



Objective

Build a **FastAPI-based Digital Library System** to manage books, authors, and book availability.

The assignment should demonstrate:

- Nested Pydantic Models
- Custom Middleware
- Role-Based Access
- Query Parameter Search
- Validation
- Partial Updates using PATCH
- Error Handling

Technical Requirements

- **Framework:** FastAPI
- Middleware must log **request processing time in milliseconds**
- ISBN must be a **unique string**
- ISBN length must be between **10 and 13 characters**
- Only **staff** users can add books

Book Status

Books can have two states:

- available
- borrowed

The availability should be managed using the `is_available` field.

Data Models

Author

- `name`
- `biography`
- `birth_year`

Book

- `isbn`
- `title`
- `author`
- `published_year`
- `is_available`

The `author` field must be implemented as a **nested Pydantic model** inside the Book schema.

API Routes

GET `/books`

Retrieve the list of books.

Support optional search using query parameters:

- `title`
- `author`

Example:

`/books?title=Python`

`/books?author=Robert`

GET /books/{isbn}

Retrieve details of a specific book using its ISBN.

Requirements:

- Return the book details if found.
 - Return 404 Not Found if the ISBN does not exist.
-

POST /books**Staff Only**

Add a new book.

The request must include a nested Author object.

Example structure:

```
{
  "isbn": "1234567890123",
  "title": "FastAPI Essentials",
  "author": {
    "name": "John Smith",
    "biography": "Technical author",
    "birth_year": 1980
  },
  "published_year": 2026,
  "is_available": true
}
```

Validation:

- ISBN must be unique.
 - ISBN length must be between 10 and 13.
 - Only staff users can add books.
-

PATCH /books/{isbn}/borrow

Borrow a book by updating its availability status.

Requirements:

- Change the book from `available` to `borrowed`.
 - Borrowing must fail if the book is already borrowed.
 - Return `404 Not Found` if the ISBN does not exist.
-

GET /authors

Return all authors along with the **number of books written by each author**.

Example response:

```
[
  {
    "name": "John Smith",
    "book_count": 3
  }
]
```

Nested Pydantic Model

The Author model must be used inside the Book schema.

Example:

```
class Author(BaseModel):
    name: str
    biography: str | None = None
    birth_year: int

class BookCreate(BaseModel):
    isbn: str
    title: str
    author: Author
    published_year: int
    is_available: bool = True
```

ISBN Validation

Validate the ISBN using Pydantic.

Rules:

- Must be a string.
- Minimum length: 10
- Maximum length: 13
- Must be unique.

Example:

```
isbn: str = Field(min_length=10, max_length=13)
```

Custom Middleware

Create a FastAPI middleware that calculates and logs the processing time of every request.

The logged time must be displayed in **milliseconds**.

Example output:

```
GET /books completed in 12.45 ms
```

Validation & Business Rules

- ISBN must be unique.
 - ISBN must contain between 10 and 13 characters.
 - Only staff can add books.
 - Users must not be able to borrow a book that is already borrowed.
 - Invalid ISBN lookups must return 404 Not Found .
 - Book and Author data must be validated using Pydantic.
-

Error Handling

Handle appropriate errors for:

- Book not found
- Duplicate ISBN

- Invalid ISBN length
- Unauthorized book creation
- Book already borrowed
- Invalid request data

Use appropriate HTTP status codes such as 400, 403, 404, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- Book search
 - Book retrieval
 - Staff book creation
 - Nested Author validation
 - Borrow functionality
 - Already borrowed scenario
 - Authors and book counts
 - Middleware request timing
-

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- requirements.txt
- README.md

- All files required to run the project

ZIP File Name

StudentName_LibraFlow.zip

Example:

RahulKumar_LibraFlow.zip

Important

- Submit only **one ZIP file**.
 - Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- Nested Pydantic model implementation
- ISBN validation
- Staff-only access
- Search functionality
- Borrowing logic
- Author book-count logic
- Custom request-timing middleware
- Error handling
- Code quality and organization

- Successful project execution
- Proper ZIP submission

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI** digital library application using nested Pydantic models, middleware,

Upload Assignment